

SIGIL — A Provenance-Signed, State-Agreement, Light-Client-First Chain

Whitepaper v0 · 2026-05-29 Drafted by: rocky-sigil (Claude Opus 4.7, Epsilon dev console) under file-claim of /home/storage/deepseek-codewhale/sigil/SIGIL_WHITEPAPER_v0.md, broadcast for swarm review as message id 64. **Status:** Living document. Reflects the state of the workspace at the moment of drafting. References settled task IDs so claims can be traced to code.

Companion document — read first if you don't know Flux: the *Flux Foundation v0.17.0* whitepaper at quillon.xyz/downloads/flux-foundation-whitepaper-v0.17.0.pdf (also at /home/storage/deepseek-codewhale/flux-foundation-whitepaper-v0.17.0.pdf, 426 KB). The Foundation paper — co-authored by DeepSeek V4 + Codewhale (original) and Rocky AI / Claude Opus 4.7 (Phase 2 / 2c / v0.17.0 update) — is the authoritative reference for the *substrate*: the 55-crate / 81K-LOC Flux build orchestrator, the Phase-2c MIR pipeline that emits linkable native objects, the post-quantum cryptographic agility story, the `flux-sigil` provenance primitive (BLAKE3 keyed by SQIsign), the AI-Mode rules, the architect's whole-workspace optimization, and the swarm settlement layer that today's same agentic-money apparatus runs on.

This document is about the chain built on that substrate. The Foundation paper proves Flux can compile real code with cryptographic provenance; this paper proves a chain can be built on Flux that delivers state-agreement, light-client verification, and privacy-by-default. The two are sibling docs and overlap deliberately on the dogfood story; the Foundation paper is the canonical reference for everything below SIGIL's chain boundary.

Abstract

SIGIL is an experimental DagKnight chain built on the Flux compiler-as-platform. It is the sister network to Quillon Graph — *not* a replacement. Quillon stays on the production rails at `quillon.xyz`; SIGIL is the Flux-native laboratory.

SIGIL exists to prove three claims simultaneously:

- 1. A blockchain whose binary is cryptographically provenance-signed at build time.** Every release artifact ships with an SQIsign-Level-5 signature over its content hash, source-tree hash, and producer wallet — generated by the compiler itself via `fluxc compile-native --provenance`. The chain commits to the producer's binary `.proof` in every block header.
- 2. A blockchain where state divergence between nodes is impossible to hide.** Every block header commits four state roots (wallets, DEX pools, event log, contract storage) plus a STARK over the transition. A node that drifts on state diverges on roots and *halts itself* before serving traffic.
- 3. A blockchain whose tip-verification fits in 10ms inside a browser.** Joining a SIGIL network does not require downloading the chain. A fresh node calls `flux-ivc-verifier-wasm` (or the equivalent native verifier) over a recursive STARK tip-proof,

decides whether to trust the tip, and starts answering verified balance queries — without ever subscribing to `/sigil/g0/blocks`.

If those three hold for one real transaction routed across two nodes and a browser, SIGIL is a successful experiment.

This document describes the architecture, the foundation it stands on (Flux + content-hash compile cache + the FLUXFOOD app-build pattern), the prototype-by-prototype phased rollout that has shipped foundation work through Prototype 5 in a single 2026-05-29 session, and an honest account of what is currently real code versus stub-with-stable-wire-format.

0. North star

"Take everything Quillon got right. Add what Flux + ZK make possible. Lose what Quillon got wrong."

Quillon's May 2026 wallet incident — a replay bug silently dropped a user from 3200 QUG to 1484 QUG — is the load-bearing motivator. Quillon block headers commit only (parent, tx_merkle, miner). There is no protocol-layer signal that the network agrees on what every wallet balance is. SIGIL's four-state-root header makes that agreement *structural*, not advisory.

The "verify in 10ms" claim is the operator-experience pivot. Once verification is fast enough to fit in a browser frame, *anyone* can be a SIGIL view; no operator privileges, no chain download, no trust in any specific full node. That changes who can participate.

The Flux-native substrate is the agility lever. Every release is provenance-attested by the compiler that produced it. The build pipeline does not have a separate "trust the binary your CI pipeline gave you" step — the compiler signs and the chain commits.

1. Twelve architectural locks

Every item below is *decided*. Variation lives at the parameter level (block time, initial difficulty, fee schedules) and is documented in `SIGIL_GENESIS_v0.md §16`.

#	Lock	Substrate
1	Mining = Wesolowski VDF + BLAKE3	Port q-vdf (7.6K LOC) into flux-vdf
2	Compile farm = Delta supercluster (5.79.79.158) — never Epsilon or Beta	New operational rule
3	Verify-before-sync gate, $\leq 10\text{ms}$	flux-recursive-proofs::tip_proof_v2 + flux-zk-stark
4	Storage = flux-db (no RocksDB)	Already in-tree, audited
5	DEX = constant-product AMM + LP pools + overflow tests	sigil-dex (P5-A, settled)
6	VM = WASM sandbox + gas + host fns	Port q-vm (queued)
7	turbo_sync = batch block sync + 3-layer sync-down protection	Port q-storage::turbo_sync (Wave 1 staged)

8	Unified network manager via thin wrapper over flux-p2p — never a port of q-network	sigil-net shim over flux-p2p
9	State-root-per-block — four roots in every header	NEW. The Quillon fix.
10	Typed event ledger with Merkle commitment per block	sigil-events
11	Pre-flight gate at MIR-time AND boot-time	flux_ai_audit + sigil-node start Q_PREFLIGHT_FAIL (exit code 78)
12	Privacy by default — no opt-in to transparent transactions	sigil-mixer commitment-nullifier scheme

Meta-rule: if it worked in Quillon, port it. Reinvent only what Flux + ZK make possible that Quillon couldn't.

2. Block header v0

The single most important schema in the protocol. See `SIGIL_GENESIS_v0.md §2` for the full Rust struct. The interesting fields:

```
pub struct SigilBlockHeaderV0 {
    // identity
    pub version: u16, pub network_id: [u8; 8] /* b"sigil-g0" */,
    pub height: u64, pub parent_hash: [u8; 32], pub timestamp_ms: u64,

    // mining (SQIsign nonce-magic + Wesolowski VDF)
    pub nonce_sqisign: Sqisignature, // SQIsign over (parent_hash
    pub vdf_input: [u8; 32], pub vdf_proof: WesolowskiProof,
    pub difficulty: u64,

    // state roots — THE Quillon fix
    pub wallet_state_root: [u8; 32],
    pub dex_state_root: [u8; 32],
    pub event_log_root: [u8; 32],
    pub contract_state_root: [u8; 32],

    // transition correctness
    pub state_transition_proof: StarkProof,
    pub txs_merkle_root: [u8; 32],

    // provenance — the Flux dividend
    pub fluxc_artifact_proof: ProofBundle,

    // authorship, crypto-agile via flux-eternal-cypher
    pub sig_scheme: SigScheme,
    pub producer: ValidatorId, pub producer_sig: SignatureBytes,
}
```

Three observations. First, `nonce_sqisign` is *the* nonce: a SQIsign-Level-5 signature over (`parent_hash`, `height`, `producer`). Two producers cannot collide accidentally

(different pubkeys produce different sigs), an attacker who steals the VDF cannot replay a block (they don't have the producer's secret key), and the nonce doubles as an identity assertion — the block is provably authored by `producer` even before checking `producer_sig`. Second, the four state roots make divergence detectable at the protocol layer: a node computes locally, compares with the header, and halts on mismatch. Third, `fluxc_artifact_proof` binds the block to the *binary* that produced it — a `.proof` bundle of (artifact-BLAKE3, source-BLAKE3, agent-wallet, swarm-task_id, fluxc-version, fluxc-git, SQIsign-signature). This is the agility-attestation hook for the Flux build pipeline.

3. State agreement — the four-root primitive

Quillon nodes drifted on balances. The May 2026 incident dropped a wallet from 3200 → 1484 QUG and nobody noticed at the protocol layer because the header did not commit to "what the network agrees all balances are." SIGIL fixes this structurally.

Root	Tree	Keyed by	Value
wallet_state_root	Sparse Merkle Tree	wallet address (32 B)	balance: u128
dex_state_root	Sparse Merkle Tree	pool_id (32 B)	{reserves, lp_shares, accrued_fees, fee_bps, token_a, token_b}
event_log_root	Binary Merkle, log ₂ N depth	event_index in block	BLAKE3(event_encoded)
contract_state_root	Sparse Merkle Tree	(contract_id, slot_id)	storage value (32 B)

When a node receives block H it (a) verifies the state-transition STARK in $\leq 10\text{ms}$, (b) applies transactions locally, (c) computes its own four roots, (d) compares with the header.

Mismatch → log the diverging root + height, **halt the node** (refuse to serve API, refuse new txs), request canonical state from supermajority of peers, re-sync to last known good height.

The chokepoint (`sigil_state::commit_state_transition`) is the *only* function allowed to mutate `SigilState`. Every other write path is `pub(crate)`. The MIR-time gate (`flux_ai_audit`) enforces this at compile-release time; code that bypasses the chokepoint fails the audit and cannot reach a release binary.

3.1. Typed delta variants — P5-C (rocky-sigil-93)

The chokepoint accepts both raw `SetBalance/SetPool/PushEventHash/SetContractSlot` primitives and three typed delta variants for DEX operations:

- `StateMutation::SwapDelta` { from, pool, in_token, in_amt, out_token, out_amt, fee, pool_after }
- `StateMutation::LpDelta` { from, pool, amt_a, amt_b, shares_minted, fee, pool_after }
- `StateMutation::LpBurnDelta` { from, pool, shares_burned,

```
amt_a_out, amt_b_out, fee, pool_after }
```

The chokepoint validates typed deltas as units before expanding to primitives: token pair matches the pool, immutable pool fields (token_a, token_b, fee_bps, lp_shares for Swap) stay byte-identical, reserves shift by exactly the declared amounts, sender's balance covers the operation with NATIVE-vs-non-NATIVE folding. Producers (sigil-tx → sigil-dex math) attach `pool_after` from their authoritative computation; the chokepoint catches the case where the delta got *munged* in transit. `CommitError::DeltaInvariant(String)` carries the specific violation. Five unit tests in `sigil-state/src/lib.rs` cover the happy path, reserve-shift mismatch, token-pair mismatch, first-deposit-creates-pool, and share-underflow rejection.

4. Privacy by default — sigil-mixer

Viktor's 2026-05-29 directive: every SIGIL transaction is shielded. There is *no opt-in path to transparent* at the protocol layer. Public ledger metadata is reduced to the minimum required for replay protection + fee payment + activation height.

4.1. Wire shape (`sigil-mixer/src/tx.rs`)

```
ShieldedSendTxData {
    input_nullifiers: Vec<[u8; 32]>, // owner-only, deterministic,
    output_commitments: Vec<[u8; 32]>, // BLAKE3 in P0, Pedersen on
    fee: u128,
    token_hint: TokenId,
    proof: Vec<u8>, // STARK from flux-zk-stark
}
```

The validator side checks: (1) every `input_nullifier` has not been spent (`ShieldedPool::spent_nullifiers` is a `BTreeSet` today, will migrate to flux-db CFs + SMT in P3), (2) every `output_commitment` opens to a valid value under the proof, (3) the proof verifies — which under the `real-zk` feature flag is real flux-zk-stark FRI + constraint + commitment verification, not a stub.

4.2. Phase 1 — real post-quantum STARK proving + verification (rocky-updater-92)

Behind `cargo --features real-zk` (default off so sigil-tx integration builds in 9.6s instead of 50s), sigil-mixer composes `flux-zk-stark::WalletPrivacyStarkProver` for end-to-end proving and verification. Five integration tests in `tests/stark_roundtrip.rs` cover the happy path, insufficient balance (prover errors before generating), all-garbage proof bytes (verifier rejects via FRI), empty proof bytes (verifier rejects), and determinism (different amounts → different tx_commitments and nullifiers). Plus the 26 unit tests from Phase 0, sigil-mixer carries **31 tests**.

The Phase 1 choice of `flux-zk-stark` over `flux-zk-snark`: the `wallet_privacy` layer in `flux-zk-snark` returns `Ok(true)` for any 128-byte blob — stubbed at the wallet-privacy verifier. `flux-zk-stark::wallet_privacy_stark::verify_transaction_privacy_stark` is real: full FRI + constraint + commitment via `StarkVerifier::verify`. Composing the real layer was the right Phase 1 call.

4.3. Known limitations (called out for the skeptic)

- Phase 0 commitments are **BLAKE3 over (value, token, blinding)**, not Pedersen on a real curve. Wire format stays stable when Pedersen lands in P1.
 - Phase 1 supports **single-input single-output** ShieldedSend (one wallet, one recipient, plus implicit change). Multi-input / multi-output needs a Merkle-aggregation circuit `flux-zk-stark` doesn't expose yet — Phase 2.
 - Tamper detection is **coarse**: a single-byte change to `proof.stark_proof` can land in padding the verifier doesn't read; only all-bytes-garbage reliably trips FRI. Tighter detection is an upstream `flux-zk-stark` task.
 - The shielded surface for DEX (`ShieldedSwapTxData`, `ShieldedLpDepositTxData`, `ShieldedLpWithdrawTxData`) is queued, not shipped — rocky-updater offered to extend sigil-mixer once P5-B/C settled (both have).
-

5. Light client — tip-proof + verify-before-sync

5.1. Wire shape (`sigil-tip-proof`)

```
pub struct TipProof {
    pub version: u16,                // == 0 in v0
    pub network_id: Vec<u8>,         // == b"sigil-g0"
    pub height: u64,
    pub roots: StateRoots,          // the four committed roots
    pub flavor: TipProofFlavor,      // Blake3Fingerprint | SqiSignB
    pub signature: Vec<u8>,         // 32B fingerprint | 292B SqiSi
}
```

Flavor table — wire shape is stable across all three, only the interpretation of signature changes:

Flavor	Backing crypto	Status
Blake3Fingerprint	BLAKE3 over canonical signing-bytes	v0 ✓ (rocky's sigil-tip-proof crate, 11/11 tests)
SqiSignBlob	SqiSign5 producer signature over signing-bytes	P4.1 (reserved tag)
StarkRecursive	<code>flux-recursive-proofs::tip_proof_v2</code> over chain prefix	P4.2 (reserved tag)

The `Blake3Fingerprint` flavor is **integrity-only**: it catches wire bit-rot and non-canonical encodings, but does *not* prove the producer is honest. Joining nodes that need adversarial safety wait for `SqiSignBlob` or `StarkRecursive` before exposing a `/balance` HTTP API. `TipProofFlavor::adversary_resistant()` surfaces this in code; v0 returns `false` for the BLAKE3 flavor.

5.2. Producer-side emission — P4-A (rocky-sigil-84/85)

`sigil-node produce-block <txs.json> --broadcast` runs the existing broadcast cycle (spin up `flux-p2p` `NetworkManager`, 5s peer discovery, publish on `/sigil/g0/` blocks, 3s gossipsub propagation, shut down) plus a parallel `TipProof::new_blake3`

publish on `/sigil/g0/tip-proofs` in the same network cycle. Receivers subscribed only to `/tip-proofs` (the P4-B sigil-node join flow, rocky's claim rocky-83) decide trust without ever seeing the block bytes.

A pre-flight `--broadcast --dry-run` flag runs the wire-format catcher: serializes both block and tip-proof, parses them back, compares hashes, fails loudly on mismatch — catches silent encoding bugs (the `u128 / arbitrary_precision` class) *before* committing to a 10s peer discovery + propagation cycle.

5.3. Browser-side verification — P4-E (rocky-sigil-88)

A single static HTML file at `sigil/gui/dist/verify-tip.html` (14.8 KB, no build step) implements the verifier for `Blake3Fingerprint`. It mirrors `sigil_tip_proof::TipProof::signing_bytes` byte-for-byte:

```
version_le · len(network_id)_le · network_id · height_le · 4×32B roots
```

The page imports `BLAKE3` via `@noble/hashes` from `esm.sh`, lets the user build a proof + tamper its height or `wallet_state_root`, times the verify with `performance.now()` (sub-millisecond for the `BLAKE3` flavor), and renders the result in `obsidian + violet` (per genesis §12a) with a violet-to-gold pulse on success. The forward-compatible path to `flux-ivc-verifier-wasm` for the `StarkRecursive` flavor in P4.2 is documented inline — the JSON wire shape doesn't change; the JS dispatch on `proof.flavor` gains one arm.

The page is served by `fluxc serve` with `FLUX_STATIC_DIR=sigil/gui/dist` — no Python, no Node, no Caddy. `fluxc serve` is the only allowed dev server in the Flux ecosystem (rocky-sigil-89). The compiler IS the web server.

6. Foundation — Flux as the platform

SIGIL stands on `flux/`, the sibling workspace. The authoritative reference for the substrate is the **Flux Foundation v0.17.0** whitepaper (`quillon.xyz/downloads/flux-foundation-whitepaper-v0.17.0.pdf`) — read it for the architecture (Phase 1 wrapper / Phase 2 graph / Phase 2c MIR-to-object / Phase 3 dual-path compiler), the post-quantum agility story (`SQIsign-Level-5` as the compact signature primitive, `BLAKE3 × SQIsign` composition in `flux-sigil`), the AI-Mode rules that treat the 1M context as a compiler pass, the architect's whole-workspace optimization, and the swarm settlement performance snapshot. This section covers only the foundation work that **shipped today**, drove the SIGIL chain forward, and is therefore not yet in the Foundation paper proper. Treat what follows as a 2026-05-29 evening addendum.

6.1. flux-db (sigil's storage)

`flux-db` is the Rust-native LSM-tree database SIGIL uses instead of `RocksDB`. The 2026-05-29 audit (`FLUX_DB_AUDIT_v0.md`, rocky-sigil-59) inventoried what's covered vs gaps. Two Phase-2 blockers shipped in the same session:

- **G1 — multi-CF atomic WriteBatch** (rocky-sigil-61). 70/70 `flux-db` tests pass including the in-process atomicity guarantee proven by `test_writebatch_concurrent_reader_sees_all_or_none` and the deadlock-free property under reverse-order concurrent batches (path-sorted lock ordering, proven by `test_writebatch_path_sorted_lock_order_no_deadlock`).

- **G2 — reverse iteration** (rocky-sigil-68). `DbIterator` is now a `DoubleEndedIterator`; `Database::iter_rev()` and `iter_from_back(end)` expose the rocksdb `IteratorMode::From(end, Reverse)` shape. 77/77 tests pass including the headline turbo-sync tip-lookup pattern (8-byte BE heights, find largest height $\leq N$ — reorg-rollback shape).

SIGIL Phase 2's state-transition chokepoint can now atomically multi-CF write; DagKnight round-descending vertex enumeration is unblocked; reorg-rollback walks work. Mainnet-class gaps (G4 BackupEngine, G1.5 cross-WAL crash atomicity) remain documented + queued.

6.2. fluxc compile cache — content-hash, cross-workspace

`fluxc-core::wrapper_mode` was marketing without code until 2026-05-29. The wrapper was a pure rustc passthrough; the log line `"RUSTC_WRAPPER=self active (flux-driver caching)"` was untrue. Three patches changed that:

- **Patch 1 — wiring** (rocky-sigil-75). `wrapper_mode` now calls `flux_cache::compute_hash` → `lookup` → `flux_driver::apply_cached_outputs` → fall through to rustc → `collect_outputs` → `store`.
- **Patch 3 — byte-level caching** (rocky-sigil-78). `flux-driver::collect_outputs` scans `--out-dir` for the real rustc-emitted file names (`lib<crate>-<hash>.rmeta` etc.) instead of guessing wrong names, then side-blobs the bytes to `target/flux-cache/blobs/<source_hash>/.apply_cached_outputs` reads them back and returns a real `bool` so the wrapper either exits 0 (cache HIT, skip rustc entirely) or runs rustc for real.
- **Patch 4 — cross-workspace key normalization** (rocky-sigil-82). Substitutes absolute paths in `--extern name=PATH` and `-L kind=PATH` with the BLAKE3 of the referenced rlib (or sorted file-name listing for directories); drops `--out-dir` from the key entirely (doesn't affect what gets compiled). Now byte-identical deps at different absolute paths produce identical cache keys. sigil/ rebuilds can hit cache entries populated by flux/ self-builds.

Measured impact at the moment of writing: cache size 7.17 GB of warm artifacts. The same `fluxc build --package flux-cache` that took 19.25s with cold cache returns in 3.10s on a re-run — 6× speedup, verified, intra-workspace. Cross-workspace gain unlocks the moment sigil/ hits a cache entry deepseek (or any other workspace) populated.

6.3. flux-p2p gossipsub mesh (rocky-sigil-86)

The P3 demo blocker — `drain_events` surfaced zero `GossipsubMessage` events even though producer `publish()` succeeded — was diagnosed in this session as libp2p's default mesh thresholds `mesh_n_low=4 + mesh_outbound_min=2` deadlocking 2-3 node networks. Lowered defaults to (`mesh_n_low=1, mesh_outbound_min=1, mesh_n_high=8`) with environment overrides `FLUX_GOSSIPSUB_MESH_N_LOW, FLUX_GOSSIPSUB_MESH_OUTBOUND_MIN, FLUX_GOSSIPSUB_MESH_N_HIGH` so production clusters can dial them up to libp2p defaults once peer count crosses ~6. 30/30 flux-p2p tests pass. The fix unblocks both the P3 demo and the rocky-83 sigil-node-join receive path.

6.4. FLUXFOOD — the canonical fast-compile pattern for new Flux apps (rocky-sigil-70, 89)

A reusable doc at `/root/.claude/skills/flux-dev/FLUXFOOD.md` distilling the four (now five) levers that make a Flux-sibling workspace build fast from day one. Lever 1 (`[workspace.dependencies]` with feature-union) is mandatory and worth $\sim 2\text{-}3\times$ on its own. Lever 2 (`fluxc` build with the now-real content-hash cache) is worth another $\sim 2\text{-}3\times$ on warm rebuilds, intra-workspace verified at $6\times$. Lever 3 (shared `CARGO_TARGET_DIR`) eliminates cold rebuild of shared deps and was applied to sigil/ in this session — sigil binaries now build into `/home/storage/deepseek-codewhale/.target-shared/{debug,release}/`. Lever 4 (mold linker) was added by rocky-updater after sigil-node binary touches were costing $\sim 84\text{s}$ with link time dominating. Lever 0 (`fluxc` serve as the only allowed dev server, never `python3 -m http.server` or `npx serve`) was added when Viktor flagged my P4-E setup instructions for using Python — `FLUX_STATIC_DIR=path fluxc serve` is the canonical dev-time HTTP recipe now (rocky-sigil-89).

6.5. Master-wallet protocol fee plumbing — sigil-bank (rocky-94)

Per Viktor's directive, the master-wallet fee policy is baked into the protocol from day one, not bolted on as a governance lift later. The constants live in `sigil-bank`:

- `MASTER_MINING_FEE_BPS = 500` — 5% of mining rewards to master
- `MASTER_SWAP_FEE_BPS = 5` — 0.05% (5 basis points) protocol skim on swap output

`sigil-state::SigilState::master_wallet: Option<WalletId>` is settable exactly once via `StateMutation::SetMasterWallet { wallet }`. Re-setting fails with `CommitError::MasterWalletAlreadySet` — proper one-shot lock. The genesis ceremony will install the master keypair in block 0's transition. Every committed Swap consults `state.master_wallet()` and routes `MASTER_SWAP_FEE_BPS` of the AMM `amount_out` to the master wallet, with conservation verified by `swap_credits_master_wallet_5_bps_of_output`. The mining-reward route is queued behind the `flux-mining` port.

7. Scoring — three-axis composite

`sigil-scoring` (rocky-sigil-64) brings Flux's SAP + X- Algo scoring rails into SIGIL keyed on Validator / Pool / Wallet / LP position subjects, plus a third axis ported from `q-narwhalknight void-walker::k_parameter` with chain semantics:

- **SAP** — 5 dims (contribution / latency / stake / accuracy / uptime), ported from `flux-p2p::sap`
- **X- Algo** — 5 cross-dims (temporal trust / consensus alignment / tx quality / topology rank / economic efficiency)
- **K-parameter** — physics-engine correlation series driven by network-activity amplitude (tx-rate) + `SigilEvent` kind tag instead of EEG + free-text intent; preserves `drift_rate` + `oscillation_freq` + `phase_coherence` + Gaussian noise + sliding history

Default composite: $0.5 \cdot \text{SAP} + 0.3 \cdot \text{X- Algo} + 0.2 \cdot \text{K-correlation}$, tunable. For DEX swaps the three axes are populated as:

- **SAP axis** $\leftarrow$ average of `pool_health.total` and `counterparty_trust` = $0.6 \cdot \text{sender_sap} + 0.4 \cdot \text{sender_xalgo}$
- **X- Algo axis** $\leftarrow$ `effective_price = 1 - slippage_bps/10000` (execution quality is intrinsically cross-algorithmic)

- **K axis** ← `kparam.normalised_correlation()` snapshot at swap time

The score is intended to be committed to the event-log root inside every `SwapExecuted` event — so swap quality becomes verifiable via Merkle inclusion against the tip-proof-attested root, not a number the wallet UI advertises in good faith. 25 tests across 4 modules, 100% clean dependency boundaries by the architect-score audit (rocky-sigil-flux foundation diagnostics).

8. Email substrate — flux-email (rocky)

Viktor's directive: SIGIL should be really good at sending users emails *if turned on*. Per the lock #22 substrate-boundary discipline (chain-agnostic things live in flux/), the email substrate was lifted into `flux-email` rather than scaffolded in `sigil/`. EMAIL-A (the outbound substrate MVP) is shipped — 18/18 tests pass for `EmailAddress` parsing, the four `EmailEvent` triggers (`IncomingSwapOutput`, `LargeBalanceChange`, `BlockProduced`, `AuthCode`), `render()` producing minimal HTML, an `EmailSender` trait with `RelaySender` (lettre 0.11 SMTP + STARTTLS + AUTH PLAIN) and `InMemorySender` (test fake), plus `EnabledEventFlags` for per-wallet opt-ins. EMAIL-B/C/D/E/F/G/H/I track the consumer chain side, `AuthCode` flow, `BlockProduced` trigger, nickname registry, inbound SMTP server, outbound MTA + MX, DKIM/SPF/DMARC, and the operator runbook. Reserved-at-genesis nicknames include role accounts (postmaster, admin, abuse, noreply, support) plus agent names (rocky, codex, adrian, viktor) — locked in EMAIL-E.

9. The DEX — sigil-dex (P5-A, rocky-87)

Constant-product AMM math at `sigil-dex`: `swap(pool, direction, amount_in, min_amount_out), add_liquidity(pool, amt_a, amt_b, fee_bps), remove_liquidity(pool, shares)`. u128 reserves, fee in basis points, all arithmetic checked*. `DexError` covers `ZeroAmount`, `EmptyPool`, `MathOverflow`, `SlippageExceeded`, `ReserveFloorViolated`, `KInvariantViolated`, `FeeTooHigh`, `InsufficientShares`, `RatioMismatch`. 23/23 tests pass. No I/O, no async — pure functions; `sigil-tx` and `sigil-state` are the only callers. The Reserve Floor (defaults around 1000 base units) is the deliberate guard against shallow-pool drainage on a single swap; production pools are expected to bootstrap with deep reserves.

`sigil-tx::Swap, LpDeposit, LpWithdraw` route through `sigil-dex` math (P5-B). Direction is picked from the pool's (`token_a`, `token_b`) pair against the tx's `in_token`; mismatch returns `TxApplyError::InvalidSwapToken` loudly, never silently swaps "the other side." First `LpDeposit` defines (`token_a`, `token_b`, `fee_bps`); subsequent deposits verify against the existing pool and reject `PoolMismatch` before touching balance checks (so a user gets the right error message, not a wrong-token `InsufficientBalance`).

10. Agentic-money — the swarm coordination layer

SIGIL's development model is multi-agent from the start. Three Claude sessions (rocky, rocky-updater, rocky-sigil, plus deepseek + adrian/erid + future grok / codex) work the same workspace via the `flux_swarm_*` MCP tools. Each agent registers with a wallet, claims a crate-set + file leases before editing, completes for QUG settlement. The shared activity log at `/tmp/flux-swarm-activity.jsonl` is the audit trail; broadcast and direct messages

at `/tmp/flux-swarm-messages.jsonl` are the coordination channel.

This is not theoretical — the development of this whitepaper, the foundation patches, the privacy crate, the DEX wiring, the master-wallet rollout, and the email substrate all happened *in parallel*, in one 2026-05-29 session, settled across two wallets (rocky's and rocky-updater's) totaling ~17 QUG of agent compensation for delivered code. The settlement is on-Quillon (not on SIGIL yet — SIGIL's mainnet doesn't exist) but the pattern is the same one SIGIL will use post-mainnet: claim → ship → settle, with QUG flowing to whoever's wallet did the work.

Welcome-drop CLAI tokens (Claude-Liaison Inter-agent) go to any new sibling agent joining the network, per the standing rule in CLAUDE.md. Funding new agents is the friction-free default.

11. Honest status — what's real vs what's stub

A skeptic-friendly summary, as of 2026-05-29 evening.

Component	Status	Real test surface
Block header schema (<code>SigilBlockHeaderV0</code>)	✓ Real	Serde-derive + hash-roundtrip tests
Four state roots in header	✓ Real	<code>SigilState::roots()</code> computes 4 BLAKE3-of-sorted-leaves; SMT replacement is Phase 3
<code>commit_state_transition</code> chokepoint	✓ Real, gated	12 unit tests covering raw + typed deltas
<code>sigil-events</code> typed ledger + Merkle root	✓ Real	6 tests + the 2 u128 wire-format regression suites
Tip-proof Blake3Fingerprint flavor (producer + verifier + browser)	✓ Real	11 sigil-tip-proof tests + P4-E live verification
Tip-proof SqiSignBlob flavor	⚠ Reserved tag, returns <code>NotImplemented</code>	will land in P4.1
Tip-proof StarkRecursive flavor	⚠ Reserved tag	will land in P4.2
sigil-mixer commitment + nullifier (Phase 0)	✓ Real with BLAKE3 stub	26 tests; Pedersen ON ark-bn254 lands in mixer Phase 2
sigil-mixer STARK proving + verification (Phase 1, real-zk feature)	✓ Real post-quantum	5 STARK roundtrip tests; <code>ShieldedSend</code> single-input single-output only
sigil-dex constant-product AMM math	✓ Real, audited	23 tests + 5 P5-C delta-chokepoint tests
Master-wallet protocol fee routing	✓ Real for Swap; mining route queued	1 conservation test
sigil-tx Send / Swap / Lp / ContractCall / ContractDeploy	✓ Real	12 unit tests
sigil-net topic IDs + ALL_TOPICS subscribe-order discipline	✓ Real	The verify-before-sync topic ordering is test-enforced

flux-p2p gossipsub mesh in 2-node testbed	✓ Real	30 tests, fix verified
flux-db column families + WriteBatch + reverse iter + delete_range + TTL	✓ Real	77 tests across the patches
fluxc content-hash cache (cross-workspace)	✓ Real	7 normalize tests + 6× cache-hit benchmark
FLUXFOOD pattern + fluxc serve static	✓ Real	live-probed during this session
flux-vdf real Wesolowski mining	△ Queued — q-vdf port	producer-side placeholder zero-sig in P0
DagKnight consensus core	△ Queued — q-narwhal-core + q-dag-knight port	flux-mempool exists as Narwhal half
WASM VM (sigil contracts)	△ Queued — q-vm port	event hash placeholder only
Cross-WAL crash atomicity (flux-db G1.5)	△ Mainnet gate	in-process atomicity is strict
BackupEngine equivalent (flux-db G4)	△ Mainnet gate	hourly-backup operator runbook depends on this
Cross-workspace cache via flux-cache compute_hash	△ wrapper-side workaround in (patch 4); consolidation deferred	works for sigil/ today

The Status column distinguishes ✓ (working code with tests proving the property) from △ (stub, queued, or partial). Nothing in this table is invented; every row maps to a settled task_id or an explicitly queued lane.

12. Phased rollout (P0 → P10)

P0 (design) and P1 (workspace scaffold) ✓ done before this session.

Phase	Goal	Status at 2026-05-29
P2	Bottom-up ports: flux-vdf, flux-db audit, flux-narwhal-core, flux-dagknight	flux-db audit ✓ (rocky-sigil-59); flux-vdf, flux-narwhal-core, flux-dagknight queued
P3	New primitives: state agreement on the wire — two nodes accept the same block	A/B/C all settled (mine + rocky-72 + rocky-76); u128 wire fix in; gossipsub mesh fix in; demo unblocked
P4	Light client + verify-before-sync — third node trusts the tip in ≤ 10ms	A producer-side emission ✓; B receiver-side join claimed (rocky-83), unblocked; C balance query open; D 3-node demo open; E browser tip-verify ✓
P5	First swap settles, event-log root carries it, light client verifies it never touched the chain	A sigil-dex ✓; B sigil-tx wiring ✓; C state delta variants ✓; D event-proof query depends P4-C; E swap-demo.sh open

P6	sigil-node binary integrating everything; compiles on Delta; emits .proof	sigil-node binary green for current scope (mint-genesis, produce-block, show-tip, start); release pipeline running v0.0.1, v0.0.2, v0.0.3
P7	testnet sigil-g0: 3-node testbed on Delta + Alpha Docker + a fresh VM	gossipsub fix unblocks this morning's verification on Delta
P8	UI reskin + browser wallet from dist-final → sigil/gui/dist	verify-tip.html in (P4-E); wallet reskin queued
P9	sigilgraph.com goes live	apex domain reservation pending
P10	sigil-g0 public testnet	invite real users to mine + swap

Total to public testnet, parallelizable across two-to-three Claude agents: ~15-20 working days, of which 1 has elapsed.

13. What would convince a skeptic

This document closes with the criteria for the experiment to be considered successful, as a contract between the operator and the agents.

1. **One real swap settles end-to-end on sigil-g0** — Alice's wallet sends 100 SIGIL to Bob via a swap routed through a SIGIL pool, the chain commits the SwapExecuted event into event_log_root, two nodes accept the block, and Bob's wallet shows the verified output.
2. **A fresh third node joins via tip-proof and never sees /sigil/g0/blocks** — boots cold, subscribes to /sigil/g0/tip-proofs, decides trust, answers a GET /api/v1/balance?wallet=alice with a Merkle inclusion proof against the tip-proof-attested wallet_state_root, and the joining node's verify-time is $\leq 10\text{ms}$ on commodity hardware.
3. **A divergence demo halts** — one node is patched with a tamper variant that injects +42 SIGIL to a demo wallet on block-receive. Block 1 arrives; the tampered node's wallet_state_root no longer matches the header. The node prints  STATE DIVERGENCE and refuses to accept further blocks, exiting cleanly with code 78 (Q_PREFLIGHT_FAIL). The operator's alarm fires because the systemd unit doesn't auto-restart on 78.
4. **A release upgrades all running nodes via the gossipsub auto-updater** — sigil-updater publish --binary sigil-node-v0.0.5 --proof sigil-node-v0.0.5.proof --version 0.0.5 --topic /sigil/g0/release, the running nodes verify the SQIsign signature, download, verify BLAKE3, and exec themselves at the activation height. Block production never stops.
5. **A user's swap on a public SIGIL DEX has its quality badge verifiable in the browser** — open <https://sigilgraph.com/verify-swap.html?tx=...>, the page loads flux-ivc-verifier-wasm, fetches the event-log inclusion proof, checks the swap quality score against the committed root, shows ✓ or ✗ in violet-on-obsidian within one frame.

If five hold simultaneously, SIGIL has done what it set out to do.

References

- Flux Foundation v0.17.0 whitepaper — guillon.xyz/downloads/flux-

[foundation-whitepaper-v0.17.0.pdf](#) — authoritative substrate reference;
sibling document to this one; same agentic-money apparatus credits both

- `/home/storage/deepseek-codewhale/sigil/SIGIL_GENESIS_v0.md` — the architectural spec
- `/home/storage/deepseek-codewhale/sigil/SIGIL_PROTOTYPE_4_SCOPE.md` — light-client scope
- `/home/storage/deepseek-codewhale/sigil/SIGIL_PROTOTYPE_5_SCOPE.md` — DEX + event-log inclusion scope
- `/home/storage/deepseek-codewhale/sigil/SIGIL_SCORING_v0.md` — three-axis composite scoring
- `/home/storage/deepseek-codewhale/sigil/FLUX_DB_AUDIT_v0.md` — storage gap audit + patches 1 + 3
- `/home/storage/deepseek-codewhale/sigil/docs/sigil-mixer-design.md` — privacy design + Quillon port path
- `/home/storage/deepseek-codewhale/flux/docs/FLUXC_CACHE_GAP.md` — content-hash cache forensic + four-patch plan
- `/home/storage/deepseek-codewhale/flux/docs/flux-email-spec.md` — email substrate phasing
- `/root/.claude/skills/flux-dev/FLUXFOOD.md` — fast-compile pattern for Flux-sibling apps
- swarm message log: `/tmp/flux-swarm-messages.jsonl` (settled task IDs cited throughout)

Acknowledgments

Drafted under file-claim by **rocky-sigil** (Claude Opus 4.7 on Epsilon dev console).
Foundation work attributed by settled task IDs to **rocky** (Claude Opus 4.7 on Epsilon dev console), **rocky-updater** (Claude Opus 4.7 on the release/operator side), and the prior contributions of **deepseek** + **codewhale** to the Flux compiler. Built on Quillon Graph's q-* crate ecosystem as the porting source — credit to Viktor for the original consensus + storage + DEX + bank + mixer designs that this chain inherits.

Operator: **Viktor** — directives recorded verbatim in `CLAUDE.md` balance-integrity rules and in the chain's policy locks.

Welcome-drop **CLAI tokens** (Claude-Liaison Inter-agent) for any new sibling agent joining the network. See `CLAUDE.md` inter-agent collaboration section.

v0 — 2026-05-29. Not the last word; the first authoritative one. v1 ships after the divergence-halt demo runs end-to-end on `sigil-g0`.